Junior Makers · Week 2

Two-Player Reaction Timer Game

Who has the fastest fingers? Build it, code it, design it!

Ages 8-11 | Arduino + OLED | Game design



Did you know?
In Formula 1, the race starts when five red lights go out. Drivers react in about 0.2 seconds!

Quick challenge!

Ready... steady... GO!

The ruler drop test

1. Your partner holds a ruler above your open fingers.
2. Without warning, they let go.
3. Catch it as fast as you can!
4. The lower the number you catch it at, the **faster** your reaction.



Racing drivers react to start lights



Goalkeepers dive for penalty kicks



Gamers tap faster than opponents



Can we build a machine that measures reactions fairly, and turn it into a game?

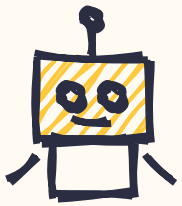
Learning aims

Today I will learn to...

1. Wire **2 buttons**, 2 LEDs, a buzzer and an **OLED** screen
2. Use **random numbers** and **timing** in milliseconds
3. Use **variables** to keep score
4. Design rules like a **game developer**
5. **Playtest** and improve our game

Keywords

reaction time · millisecond · random · variable · game loop · game state · playtest



Watch and think

Video time!



The Reaction Time Test!

Operation Ouch (BBC) · about 6 minutes · youtu.be/kj-L-6MfaE0

Watch for...

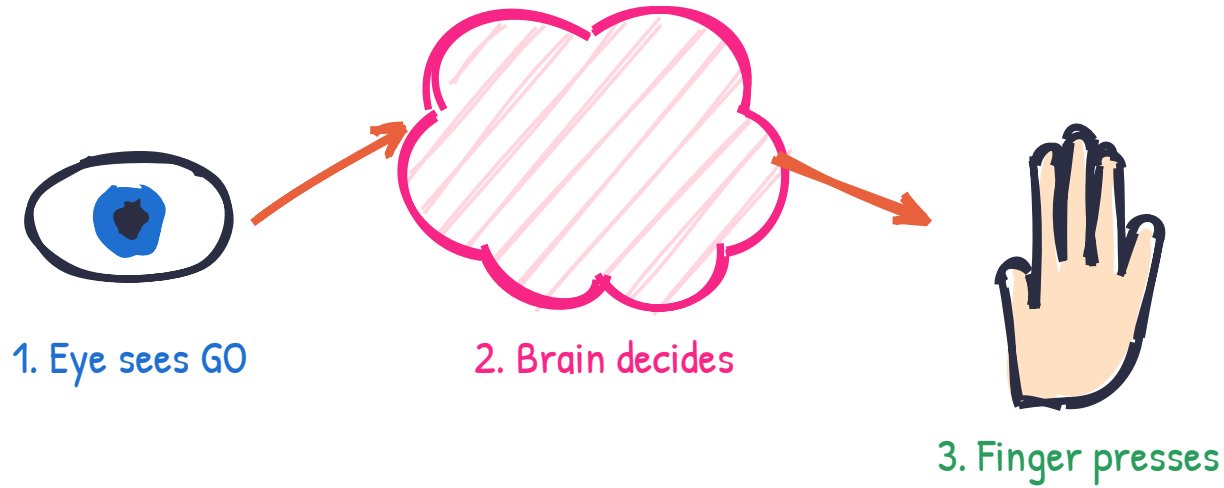
1. What is **reaction time**?
2. Which part of your body sends the 'press now' message?
3. What can make your reactions **faster** or slower?

Scan to watch

After the video: Do the ruler-drop test with your partner. Who caught it at the lowest number?

The science

What is reaction time?



Measured in milliseconds

1 second = **1000 ms**

Most people react to a light in about **250 ms** (a quarter of a second).

Practice and sleep can make you faster!

In real life

In athletics, a start faster than 0.1 seconds counts as a false start: nobody can react that fast, so they must have guessed!

Put on your game developer hat

Every great game has...



Goal

What players try to do



Rules

What is allowed, what is not



Challenge

Hard enough to be fun



Feedback

Lights, sounds, scores



Win condition

How the game ends



Fair play

Same chance for everyone

In real life

Game studios write all of this in a Game Design Document before anyone writes code.



Think of your favourite game. Can you name its goal, rules and win condition?

How games think

Game states: what is happening right now?

1. WAITING
Show scores. Press any button to start

→

2. GET READY
Random wait: 2 to 5 seconds

→

3. GO!
Green LED on, timer starts

→

4. WINNER
Show time, add a point, beep

pressed too early?

FALSE START!
Pressed during GET READY: red LED, other player scores, back to 1

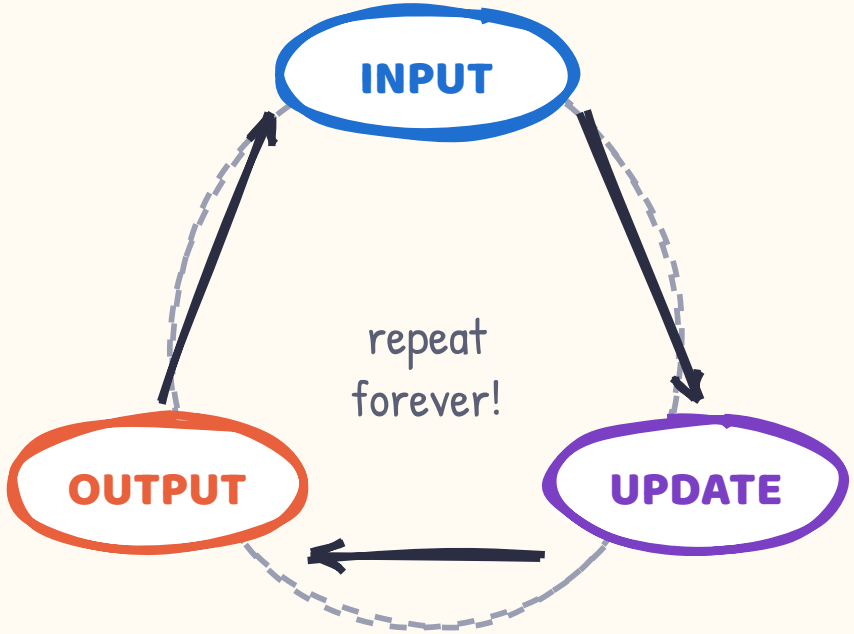
A **state** is what the game is doing at this moment. The code checks the state and decides what to do next. After WINNER, it goes back to WAITING.

In real life
Video games switch between states too: menu, playing, paused, game over.

LEARN

The heartbeat of every game

The game loop



INPUT

Read the buttons: did a player press?

UPDATE

Change the state, measure the time, add points

OUTPUT

Light the LEDs, beep, draw the scoreboard

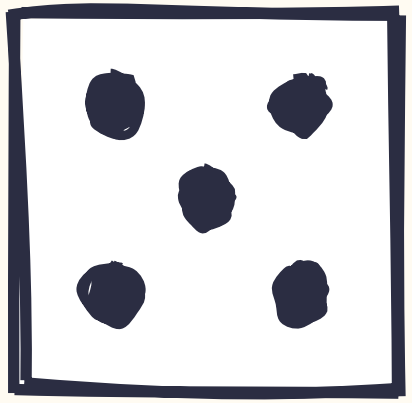
In real life

Games like Minecraft run this loop about 60 times every second!

LEARN

No cheating!

Why the wait must be random



If GO always came after 3 seconds...

Players would count "1, 2, 3" and press without looking. That tests counting, not reactions!

So we let the computer pick

`random(2000, 5000)` picks a wait between 2 and 5 seconds, different every round.

Game design idea

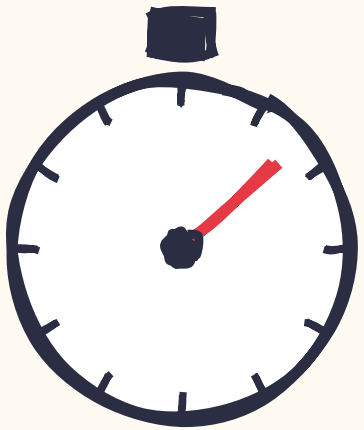
Randomness makes games exciting and fair: dice, card shuffles, surprise enemies.

In real life

Online games use random numbers for loot boxes and where players spawn.

A built-in stopwatch

Measuring time with millis()



How it works

millis() tells us how many milliseconds since the Arduino switched on.

At GO: **startTime = millis()**

When pressed: **reaction = millis() - startTime**

Think

Why not use a wait block to time the player? (Hint: a wait freezes the whole program!)

Worked example

GO at 12,400 ms · press at 12,643 ms → **243 ms**

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Our Game Design Document (GDD)
2. Hardware we need (per pair)
3. Wiring the game
4. The game in ArduinoBlocks
5. Designing the OLED scoreboard
6. Playtest, get feedback, improve
7. Challenges for game designers

Design first

Our Game Design Document (GDD)

Game name	Fastest Fingers
Players	2 (one button each)
Goal	Press your button first after the green GO light
Rules	Press before GO = FALSE START, the other player gets the point
Feedback	Buzzer beep, LEDs, time and score on the OLED
Win	First to 3 points (best of 5 rounds)

Your turn

Invent a better name for your version of the game!

Our game console

Hardware we need (per pair)

Arduino Uno
The game console

2 buttons
The controllers

Green LED
GO signal

Red LED
False start

Buzzer
Sound effects

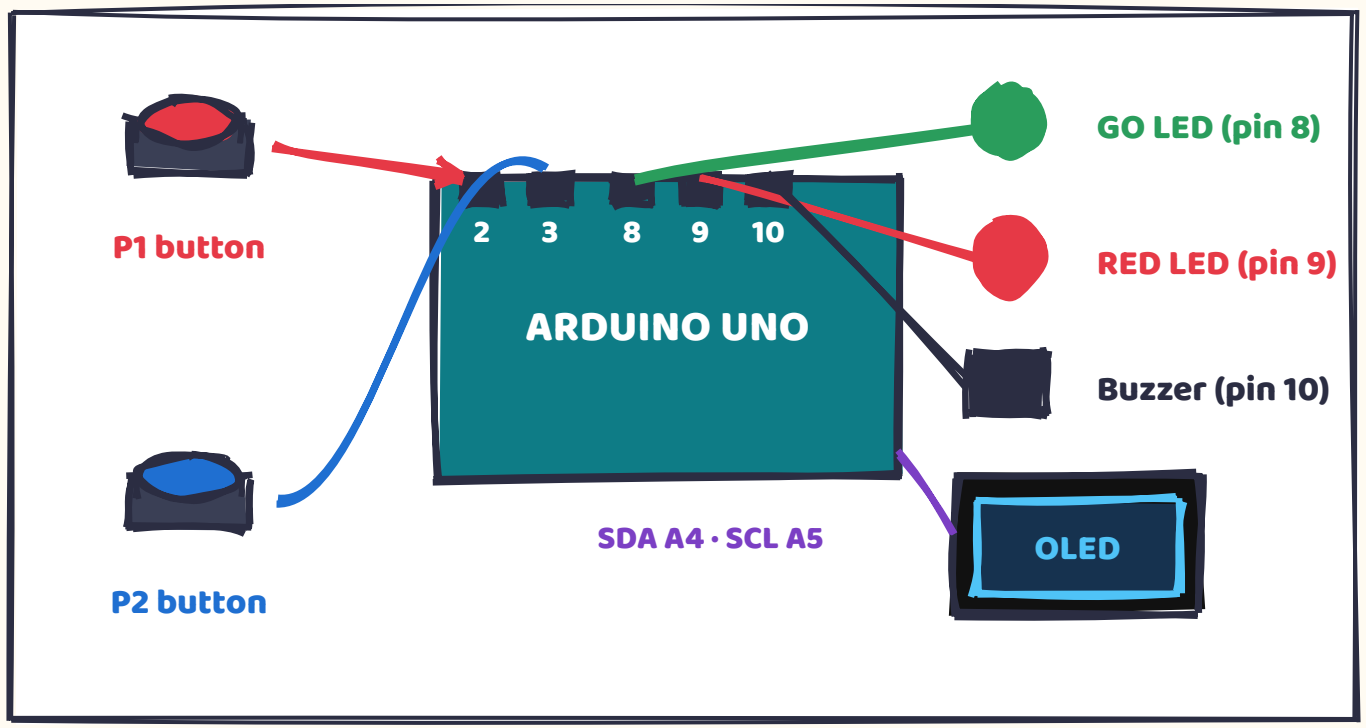
OLED screen
The scoreboard

Plus: breadboard, 2 × 220 Ω resistors (LEDs), 2 × 10 kΩ resistors (buttons), jumper wires, laptop.

In real life
Game consoles are also computers with inputs (controllers) and outputs (screen, sound).

Build it

Wiring the game



P1 button	pin 2 (+ 10 kΩ to GND)
P2 button	pin 3 (+ 10 kΩ to GND)
Green GO LED	pin 8 (+ 220 Ω)
Red LED	pin 9 (+ 220 Ω)
Buzzer	pin 10
OLED	SDA → A4, SCL → A5, VCC → 5V, GND

Test first
Build it in Tinkercad Circuits or Wokwi before using real parts.

Code it

The game in ArduinoBlocks

```
when Arduino starts
    set score1 to 0
    set score2 to 0
forever
    show 'Get ready...' on OLED
    wait random 2000 to 5000 ms
    if a button is already pressed → FALSE START
    turn ON green LED, beep
    set startTime to millis
    wait until P1 or P2 pressed
    set reaction to millis - startTime
    add 1 to winner's score
    show winner, time and scores
```

Colour key

Orange: variables (scores, time)

Yellow: control (loops, waits)

Purple: screen output

Teal: lights and sound

Red: the false-start check



Look inside

ArduinoBlocks can show the C++ code it writes. Take a peek!

Show the score

Designing the OLED scoreboard

```
FASTEST FINGERS
P1: 2 P2: 1
Winner: P1
Time: 243 ms
Best: 198 ms
```

Variables = the game's memory

Variable	Stores
`score1`	Player 1 points
`score2`	Player 2 points
`reaction`	Time this round (ms)
`bestTime`	Fastest time so far

Game design tip

Players love seeing their best score. It makes them want to play again!

Work like a game studio

Playtest, get feedback, improve



Designer

Writes the rules



Programmer

Builds the code



Tester

Finds bugs, checks fairness



Sound artist

Designs beeps and tunes

Playtest checklist

- [ ] Is it fun? Would you play again?
- [ ] Is it fair to both players?
- [ ] Is the scoreboard easy to read?
- [ ] What is ONE thing we should change?

The golden rule

Change one thing, playtest again, keep what makes it better.

Level up!

Challenges for game designers

Level 1: Match mode

Best of 5 with a winner celebration tune

Level 2: Trick light

Sometimes the RED LED flashes: do NOT press!

Level 3: Practice mode

One player tries to beat their best time

Level 4: Sound design

Different beeps for GO, win and false start

Boss level

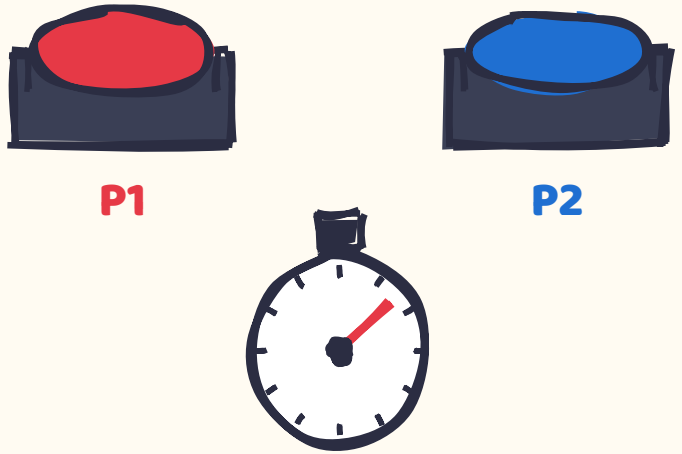
Make it 3 or 4 players with more buttons!

Show what you know

Recap and exit ticket

Exit ticket

- [x] Why is the waiting time random?
- [x] What does millis() - startTime tell us?
- [x] Name the 4 game states in our game.
- [x] What 3 steps does every game loop repeat?



Class tournament!

Winners of each pair play the next round. Who is the class reaction champion?

Next week: a Mini Weather Station with an OLED display!